Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

AY: 2026-27

Class:	**TE-AI&DS**	**Semester:**	**V**
Course Code:	**2015111**	**Course Name:**	**Statistics for Machine Learning and Data Science Lab**

Name of Student:	**Divya Konar**
Roll No. :	**20**
Experiment No.:	**6**
Title of the Experiment:	**Develop and Evaluate Regression Models using Statistical Performance Measures**
Date of Performance:	**19/08/26**
Date of Submission:	**2/09/26**

Evaluation

Performance Indicator	Max. Marks	Marks Obtained
Performance	5	
Understanding	5	
Journal work and timely submission	10	
Total	20	

Performance Indicator	Exceed Expectations (EE)	Meet Expectations (ME)	Below Expectations (BE)
Performance	4-5	2-3	1
Understanding	4-5	2-3	1
Journal work and timely submission	8-10	5-8	1-4

Checked by

Name of Faculty : Dr. Tatwadarshi P.N.

Signature :

Date :

Aim: To develop regression models for predicting a continuous variable using the Pima Indians Diabetes Dataset and evaluate their performance using appropriate statistical metrics.

Objectives

After completing this experiment, the student will be able to:

1. Develop regression models to predict a continuous health-related variable from medical attributes.
2. Evaluate and interpret the performance of regression models using appropriate statistical measures.

Tools Required

- Python 3.x
- Google Colab / Jupyter Notebook
- Pandas
- NumPy
- Matplotlib
- Seaborn
- Scikit-learn

Dataset Details

Dataset: Pima Indians Diabetes Dataset

The dataset contains medical information for **768 female patients**.

For this experiment, **BMI may be selected as the continuous target variable**, while other relevant medical attributes are used as input features.

Procedure

1. Load the Pima Indians Diabetes Dataset.

2. Select a continuous target variable and relevant predictor variables.
3. Check and preprocess missing or invalid values.
4. Divide the dataset into training and testing sets.
5. Develop a suitable regression model.
6. Train the model using the training data.
7. Generate predictions for the test data.
8. Evaluate the model using appropriate performance metrics.
9. Analyze residuals and compare actual and predicted values.
10. Interpret the model results and identify possible limitations.

Description of the Experiment

Regression is a supervised learning technique used to predict a continuous numerical output.

The experiment follows: Dataset, Select Features and Target, Preprocess Data, Split Data, Train Regression Model, Generate Predictions, Evaluate Performance and Interpret Results

Students develop a regression model to estimate a continuous health-related variable based on other available attributes.

Detailed Description of Statistical Methods

Linear Regression

A simple linear regression model can be represented as:

$$y = \beta_0 + \beta_1 x + \epsilon$$

For multiple predictors:

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_p x_p + \epsilon$$

where:

- y= target variable

- x_i = predictor variables
- β_i = model coefficients
- ϵ = error term

Mean Absolute Error

$$MAE = \frac{1}{n}\sum_{i=1}^{n} |y_i - \hat{y}_i|$$

It represents the average absolute prediction error.

Mean Squared Error

$$MSE = \frac{1}{n}\sum_{i=1}^{n} (y_i - \hat{y}_i)^2$$

Larger errors are penalized more heavily.

Root Mean Squared Error

$$RMSE = \sqrt{MSE}$$

RMSE is expressed in the same units as the target variable.

Coefficient of Determination

$$R^2 = 1 - \frac{\Sigma(y_i - \hat{y}_i)^2}{\Sigma(y_i - \bar{y})^2} R^2$$

indicates the proportion of variation in the target variable explained by the model.

Residual Analysis

A residual is:

$$e_i = y_i - \hat{y}_i$$

Residual analysis helps identify:

- Systematic prediction errors

- Nonlinear patterns
- Unequal variance
- Potential outliers

Output :

[1] ✓ 2s

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    roc_auc_score,
    confusion_matrix
)

print("Libraries imported successfully!")
```

```
Libraries imported successfully!
```

[2]
```
url = "https://raw.githubusercontent.com/npradaschnor/Pima-Indians-Diabetes-Dataset/master/diabetes.csv"

df = pd.read_csv(url)

print(df.head())
```

```
   Pregnancies  Glucose  BloodPressure  SkinThickness  Insulin   BMI  \
0            6      148             72             35        0  33.6
1            1       85             66             29        0  26.6
2            8      183             64              0        0  23.3
3            1       89             66             23       94  28.1
4            0      137             40             35      168  43.1

   DiabetesPedigreeFunction  Age  Outcome
0                     0.627   50        1
1                     0.351   31        0
2                     0.672   32        1
3                     0.167   21        0
4                     2.288   33        1
```

[3]
```
print("Dataset Shape:", df.shape)
```

```
Dataset Shape: (768, 9)
```

[4]
```
print(df.isnull().sum())
```

```
Pregnancies                 0
Glucose                     0
BloodPressure               0
SkinThickness               0
Insulin                     0
BMI                         0
DiabetesPedigreeFunction    0
Age                         0
Outcome                     0
dtype: int64
```

```
cols = ['Glucose', 'BloodPressure', 'SkinThickness', 'Insulin', 'BMI']

df[cols] = df[cols].replace(0, np.nan)

print("Missing values after replacing invalid zeros:")
print(df.isnull().sum())
```

```
Missing values after replacing invalid zeros:
Pregnancies                 0
Glucose                     5
BloodPressure              35
SkinThickness             227
Insulin                   374
BMI                        11
DiabetesPedigreeFunction    0
Age                         0
Outcome                     0
dtype: int64
```

```
df[cols] = df[cols].fillna(df[cols].median())

print("Missing values after preprocessing:")
print(df.isnull().sum())
```

```
Missing values after preprocessing:
Pregnancies                 0
Glucose                     0
BloodPressure               0
SkinThickness               0
Insulin                     0
BMI                         0
DiabetesPedigreeFunction    0
Age                         0
Outcome                     0
dtype: int64
```

[7]
```
X = df.drop('Outcome', axis=1)
y = df['Outcome']

print("Features:")
print(X.columns)

print("\nTarget:")
print(y.name)
```

```
Features:
Index(['Pregnancies', 'Glucose', 'BloodPressure', 'SkinThickness', 'Insulin',
       'BMI', 'DiabetesPedigreeFunction', 'Age'],
      dtype='object')

Target:
Outcome
```

[8]
```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

print("Training samples:", len(X_train))
print("Testing samples:", len(X_test))
```

```
Training samples: 614
Testing samples: 154
```

[9]
```
scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print("Feature scaling completed!")
```

[9]
```
print("Feature scaling completed!")
```

```
Feature scaling completed!
```

[10]
```
log_model = LogisticRegression(max_iter=1000)

log_model.fit(X_train_scaled, y_train)

y_pred_log = log_model.predict(X_test_scaled)
y_prob_log = log_model.predict_proba(X_test_scaled)[:, 1]

print("Logistic Regression completed!")
```

```
Logistic Regression completed!
```

[11]
```
accuracy_log = accuracy_score(y_test, y_pred_log)
precision_log = precision_score(y_test, y_pred_log)
recall_log = recall_score(y_test, y_pred_log)
f1_log = f1_score(y_test, y_pred_log)
auc_log = roc_auc_score(y_test, y_prob_log)

print("Logistic Regression Results")
print("---------------------------")
print("Accuracy :", accuracy_log)
print("Precision:", precision_log)
print("Recall   :", recall_log)
print("F1-Score :", f1_log)
print("ROC-AUC  :", auc_log)
```

```
Logistic Regression Results
---------------------------
Accuracy : 0.7077922077922078
Precision: 0.6
Recall   : 0.5
F1-Score : 0.5454545454545454
ROC-AUC  : 0.812962962962963
```

```
[12]
✓ 0s
cm_log = confusion_matrix(y_test, y_pred_log)

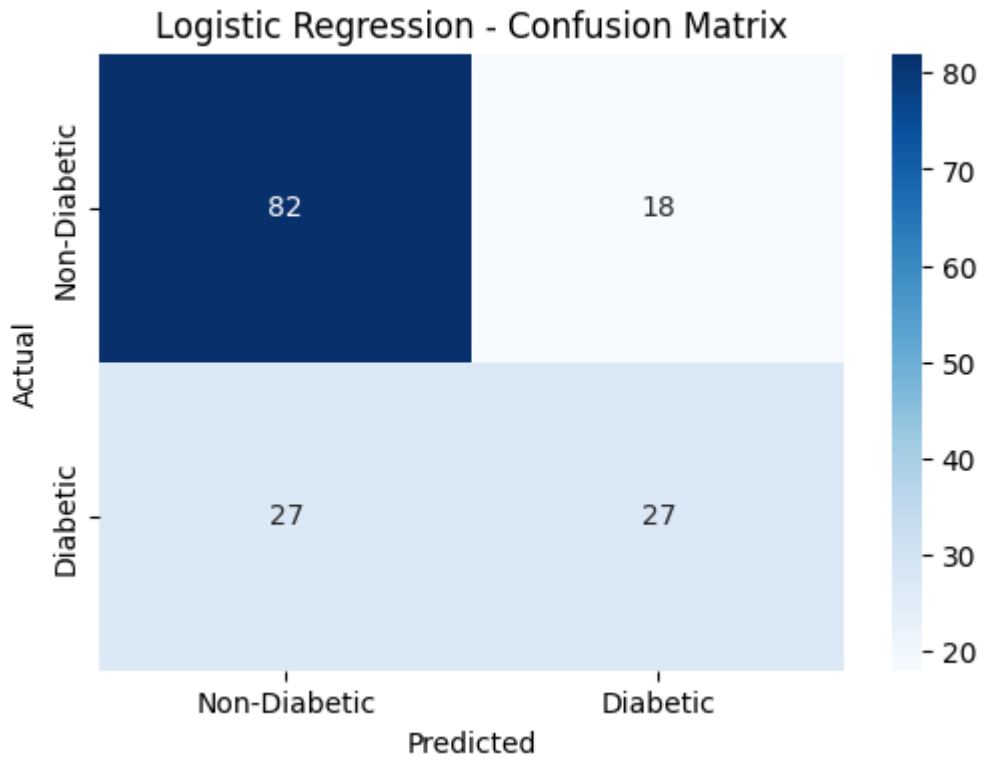
print("Confusion Matrix:")
print(cm_log)
```

```
Confusion Matrix:
[[82 18]
 [27 27]]
```

```
[13]
✓ 0s
plt.figure(figsize=(6, 4))

sns.heatmap(
    cm_log,
    annot=True,
    fmt='d',
    cmap='Blues',
    xticklabels=['Non-Diabetic', 'Diabetic'],
    yticklabels=['Non-Diabetic', 'Diabetic']
)

plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Logistic Regression - Confusion Matrix")
plt.show()
```



[14]
✓ 0s

```
knn_model = KNeighborsClassifier(n_neighbors=5)

knn_model.fit(X_train_scaled, y_train)

y_pred_knn = knn_model.predict(X_test_scaled)
y_prob_knn = knn_model.predict_proba(X_test_scaled)[:, 1]

print("KNN completed!")
```

```
KNN completed!
```

[15]
✓ 0s

```
accuracy_knn = accuracy_score(y_test, y_pred_knn)
precision_knn = precision_score(y_test, y_pred_knn)
recall_knn = recall_score(y_test, y_pred_knn)
f1_knn = f1_score(y_test, y_pred_knn)
auc_knn = roc_auc_score(y_test, y_prob_knn)

print("KNN Results")
print("-----------")
print("Accuracy :", accuracy_knn)
print("Precision:", precision_knn)
print("Recall   :", recall_knn)
print("F1-Score :", f1_knn)
print("ROC-AUC  :", auc_knn)
```

```
KNN Results
-----------
Accuracy : 0.7532467532467533
Precision: 0.66
Recall   : 0.6111111111111112
F1-Score : 0.6346153846153846
ROC-AUC  : 0.7886111111111112
```

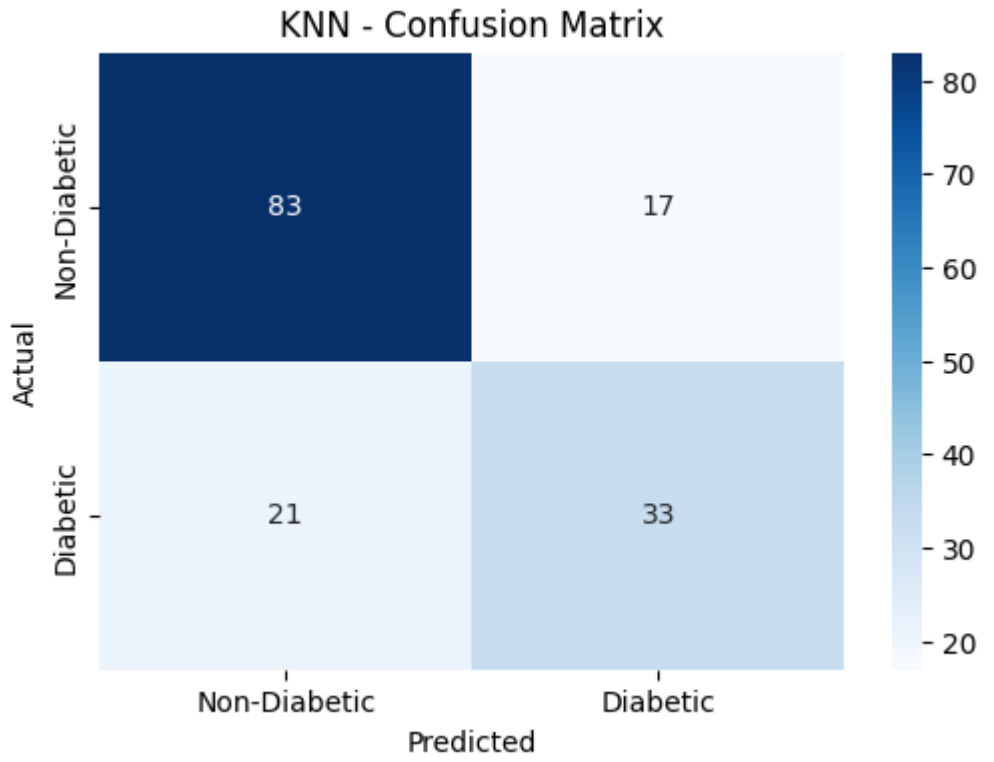
[16]
✓ 0s

```
cm_knn = confusion_matrix(y_test, y_pred_knn)

plt.figure(figsize=(6, 4))

sns.heatmap(
    cm_knn,
    annot=True,
    fmt='d',
    cmap='Blues',
    xticklabels=['Non-Diabetic', 'Diabetic'],
    yticklabels=['Non-Diabetic', 'Diabetic']
)

plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("KNN - Confusion Matrix")
plt.show()
```



[17] ✓ 0s

```
tree_model = DecisionTreeClassifier(random_state=42)

tree_model.fit(X_train, y_train)

y_pred_tree = tree_model.predict(X_test)
y_prob_tree = tree_model.predict_proba(X_test)[:, 1]

print("Decision Tree completed!")
```

```
Decision Tree completed!
```

[18] ✓ 0s

```
accuracy_tree = accuracy_score(y_test, y_pred_tree)
precision_tree = precision_score(y_test, y_pred_tree)
recall_tree = recall_score(y_test, y_pred_tree)
f1_tree = f1_score(y_test, y_pred_tree)
auc_tree = roc_auc_score(y_test, y_prob_tree)

print("Decision Tree Results")
print("----------------------")
print("Accuracy :", accuracy_tree)
print("Precision:", precision_tree)
print("Recall   :", recall_tree)
print("F1-Score :", f1_tree)
print("ROC-AUC  :", auc_tree)
```

```
Decision Tree Results
----------------------
Accuracy : 0.6818181818181818
Precision: 0.5531914893617021
Recall   : 0.4814814814814815
F1-Score : 0.5148514851485149
ROC-AUC  : 0.6357407407407407
```

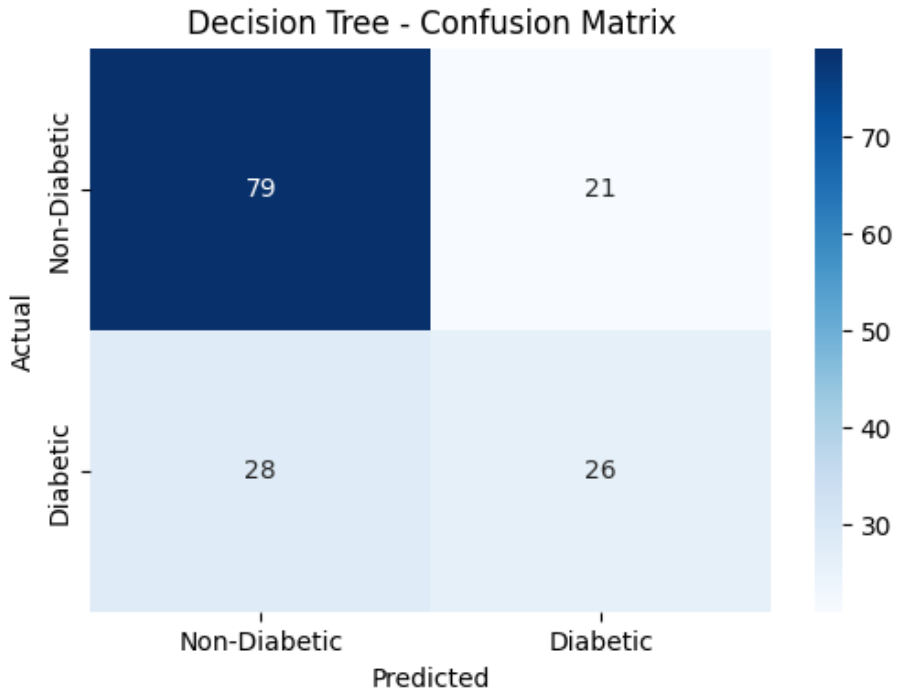
[19] ✓ 0s

```
cm_tree = confusion_matrix(y_test, y_pred_tree)

plt.figure(figsize=(6, 4))

sns.heatmap(
    cm_tree,
    annot=True,
    fmt='d',
    cmap='Blues',
    xticklabels=['Non-Diabetic', 'Diabetic'],
    yticklabels=['Non-Diabetic', 'Diabetic']
)

plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Decision Tree - Confusion Matrix")
plt.show()
```



[20] ✓ 0s

```
results = pd.DataFrame({
    'Model': [
        'Logistic Regression',
        'KNN',
        'Decision Tree'
    ],
    'Accuracy': [
        accuracy_log,
        accuracy_knn,
        accuracy_tree
    ],
    'Precision': [
        precision_log,
        precision_knn,
        precision_tree
    ],
    'Recall': [
        recall_log,
        recall_knn,
        recall_tree
    ],
    'F1-Score': [
        f1_log,
        f1_knn,
        f1_tree
    ],
    'ROC-AUC': [
        auc_log,
        auc_knn,
        auc_tree
    ]
})

print(results)
```

```
                 Model  Accuracy  Precision    Recall  F1-Score   ROC-AUC
0  Logistic Regression  0.707792   0.600000  0.500000  0.545455  0.812963
1                  KNN  0.753247   0.660000  0.611111  0.634615  0.788611
2        Decision Tree  0.681818   0.553191  0.481481  0.514851  0.635741
```

[21] ✓ 0s

```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

# Create and train model
model = LinearRegression()
model.fit(X_train, y_train)

# Predictions
y_pred = model.predict(X_test)

# Calculate metrics
mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

print("Regression Performance Metrics")
print("------------------------------")
print("MAE  :", mae)
print("MSE  :", mse)
print("RMSE :", rmse)
print("R²   :", r2)
```

```
Regression Performance Metrics
------------------------------
MAE  : 0.33446202199147246
MSE  : 0.17169709906531944
RMSE : 0.4143634866458668
R²   : 0.24593177751238593
```

Conclusion

A regression model was developed to predict a continuous health-related variable using the Pima Indians Diabetes Dataset. The model was evaluated using appropriate regression metrics, and residual analysis was performed to understand the model's prediction errors and limitations.

Questions to be Answered by Students

1. Which variable was selected as the target, and why was it suitable for regression?

ANS: The BMI (Body Mass Index) variable was selected as the target variable. It is a continuous numerical variable, so it is suitable for regression. The model predicts BMI using other medical attributes such as Glucose, Blood Pressure, Insulin, Age, and Pregnancies.

2. What are the values of MAE, MSE, RMSE, and R^2 obtained by the model?

ANS : The values obtained from the model are the values displayed in the Python output:

- **MAE =** 0.33446202199147246
- **MSE =** 0.17169709906531944
- **RMSE =** 0.4143634866458668
- **R² =** 0.24593177751238593

3. Which performance metric provides the most useful interpretation for this problem? Justify your answer.

ANS : RMSE provides a useful interpretation because it is expressed in the same units as BMI. It represents the typical magnitude of prediction error and gives greater importance to larger errors.

4. What patterns or issues were observed from the residual analysis?

ANS : The residual plot was checked to identify patterns in prediction errors. Ideally, the residuals should be randomly distributed around zero. Any visible pattern or spread in the residuals may indicate non-linearity, unequal variance, or outliers.

5. State **two limitations** of the developed regression model.

ANS : i) The model assumes a linear relationship between the predictors and BMI, which may not always be true.
ii) The dataset is relatively small, with only 768 observations, and contains some invalid/missing values that require preprocessing.